



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



CARÁTULA PARA ENTREGA DE PRÁCTICAS

Profesor:	Dr. Sergio Teodoro Vite
Grupo de Laboratorio:	06
Número de práctica:	05
Título de la práctica:	Adaptación y Carga de Modelos
Fecha de entrega:	06 de octubre de 2026

Número de cuenta	Nombre completo	Grupo de teoría
321137135	Basilio Illescas Andrés	04
424065465	Cruz Macedo Samuel Santiago	04
321258173	Medel Piña Ángel de Jesús	02
321150503	Pérez Olvera Alexis Abraham	04
320224159	Segura Cedeño Luisa María	02

HOJA DE EVIDENCIAS

PRÁCTICA 05: ADAPTACIÓN Y CARGA DE MODELOS

Lista de números de cuenta exclusivamente:

321137135

424065465

321258173

321150503

320224159

1. Resumen

Resumen

En esta práctica trabajamos en la adaptación y carga de modelos 3D complejos para construir una escena interactiva utilizando C++ y OpenGL 3.3 en perfil núcleo. Para importar e interpretar archivos externos (como FBX u OBJ), nos apoyamos en la biblioteca Assimp y diseñamos una arquitectura de clases para gestionar mallas (Mesh) y modelos (Model), enviando la geometría, texturas y normales a la GPU mediante VBOs y VAOs. Como variación de la actividad, se aplicó el modificador Array en Blender para duplicar elementos arquitectónicos y expandir la escala de la ciudad de forma eficiente. Finalmente, se integraron personajes con animaciones controladas mediante entradas del teclado, logrando un escenario fluido y optimizado gracias al manejo jerárquico de las mallas y la memoria gráfica.

Palabras clave: Carga de modelos, Assimp, mallas 3D, OpenGL, Blender Array, animaciones, VAO/VBO.

Abstract

In this lab session, we worked on adapting and loading complex 3D models to build an interactive scene using C++ and OpenGL 3.3 core profile. To import and parse external file formats (such as FBX or OBJ), we relied on the Assimp library and designed a class architecture to manage meshes and models, sending geometry, textures, and normals to the GPU via VBOs and VAOs. As a key variation of the activity, Blender's Array modifier was applied to duplicate architectural elements and efficiently scale up the city environment. Finally, animated characters controlled via keyboard inputs were integrated, achieving a fluid and optimized scene through hierarchical mesh organization and efficient GPU memory management.

Keywords: Model loading, Assimp, 3D meshes, OpenGL, Blender Array, animations, VAO/VBO.

2. Introducción

Hoy en día, el cómputo gráfico se apoya por completo en la capacidad de procesar y desplegar escenas 2D y 3D de forma rápida y eficiente a través del cauce de renderizado (*pipeline* gráfico) acelerado por hardware [1]. En este terreno, herramientas como OpenGL destacan por ofrecer una interfaz estándar y muy robusta que nos permite comunicarnos directamente con la tarjeta gráfica o GPU [4].

Como señalan [5], dominar las bases matemáticas y las transformaciones geométricas es esencial si queremos construir entornos tridimensionales interactivos. En esta práctica abordamos todo el flujo necesario para importar, adaptar y renderizar mallas 3D complejas que fueron diseñadas previamente en software de modelado externo como Blender. Para lograr que estos objetos se integren de forma natural en la escena, es indispensable entender bien tanto la estructura de la geometría como la manera en que se gestionan los recursos en la memoria de la GPU.

Para proyectar objetos en un espacio tridimensional, el *pipeline* moderno reemplaza las funciones fijas de antes por etapas programables mediante *shaders* [3]. Cuando cargamos un modelo desde un archivo externo (por ejemplo, en formato FBX u OBJ), su información se divide en vértices, normales, coordenadas de textura y caras. Todos estos datos se envían a la memoria gráfica mediante objetos de búfer de vértices (VBO, *Vertex Buffer Objects*) y se organizan mediante objetos de arreglos de vértices (VAO, *Vertex Array Objects*), los cuales guardan la configuración de los atributos necesarios para el dibujado.

En la etapa del *Vertex Shader*, cada vértice pasa de su espacio local al espacio de recorte mediante la aplicación encadenada de matrices de transformación, conocidas como el producto Modelo-Vista-Proyección (MVP) [2]:

$$v_{\text{clip}} = P \cdot V \cdot M \cdot v_{\text{local}} \quad (1)$$

En esta ecuación, la matriz de modelo (M) nos ayuda a posicionar, escalar y orientar la malla en la escena; la matriz de vista (V) define la posición y orientación de la cámara; y la matriz de proyección (P) le da el efecto de perspectiva dentro del volumen de visión.

Después, la fase de rasterización convierte las primitivas geométricas proyectadas en un conjunto de fragmentos que corresponden a los píxeles en pantalla. Finalmente, el *Fragment Shader* se encarga de definir el color de cada píxel combinando mapas de textura, propiedades del material e iluminación [3].

Para facilitar el manejo de mallas complejas y escenarios extensos, recurrimos a librerías de importación como Assimp (*Open Asset Import Library*). Esta herramienta nos permite leer estructuras jerárquicas de nodos, extraer submallas (*submeshes*) y vincular sus respectivas texturas. De este modo, logramos separar el trabajo de diseño artístico de la programación del motor gráfico, optimizando la carga de trabajo y garantizando un rendimiento fluido en tiempo real.

3. Metodología

3.1. Configuración del entorno de trabajo

Para el desarrollo de la práctica se configuró un entorno de trabajo en C++ enfocado en el desarrollo de gráficos 3D en tiempo real. Se integraron bibliotecas especiales para la gestión de ventanas, operaciones matemáticas vectoriales, procesamiento de imágenes y la carga de modelos tridimensionales complejos creados previamente en herramientas de modelado externo. En la Tabla 1 se detallan los elementos de software, bibliotecas y herramientas empleadas.

Elemento	Especificación / Versión empleada
Sistema operativo	Windows 11 / Linux (64 bits)
Entorno de desarrollo	Visual Studio Code / Microsoft Visual Studio
Compilador y estándar	GCC / MSVC (soporte para C++17 / C++20)
Software de diseño 3D	Blender (modelado y modificador <i>Array</i>)
API gráfica	OpenGL 3.3 (Perfil Core)
Bibliotecas principales	GLFW (gestión de ventanas e interacción por teclado), GLAD (cargador de funciones de OpenGL), GLM (álgebra lineal y matrices MVP), Assimp (importación de mallas 3D), <code>stb_image.h</code> (carga de mapas de textura)
Hardware de prueba	GPU con soporte nativo para OpenGL 3.3 Core

Tabla 1: Requerimientos de hardware, software y bibliotecas utilizadas.

3.2. Actividad 1: Construcción e Integración del Escenario Completo 3D

Para esta práctica, el objetivo fue armar un escenario tridimensional desde cero. Primero diseñamos el edificio y sus alrededores en Blender. Una vez que quedó listo, exportamos todo a un archivo FBX para poder meterlo a nuestro programa en C++ y renderizarlo usando OpenGL con la ayuda de la librería Assimp.

3.2.1. Herramientas y programas que utilizamos

Para que todo esto funcionara, necesitamos juntar varias librerías y herramientas para cargar, procesar y mostrar los modelos en 3D:

- **Blender:** Fue la herramienta para modelar. Aquí creamos las formas en 3D, le pusimos los materiales y exportamos la escena completa en formato `.fbx`.
- **Assimp (Open Asset Import Library):** Esta librería se encarga de leer el archivo `.fbx` y traducir toda esa información en estructuras que nuestro código en C++ pueda entender.
- **GLFW:** Nos sirvió para crear la ventana donde se muestra todo, además de capturar las entradas del teclado y del ratón para movernos por el escenario.
- **GLEW (OpenGL Extension Wrangler Library):** La usamos para conectar y activar las funciones nativas de OpenGL en nuestro sistema.
- **GLM (OpenGL Mathematics):** Nos facilitó las matemáticas para hacer las matrices y vectores de las transformaciones, la cámara y las proyecciones.
- **OpenGL (v3.3 Core Profile):** Es la API gráfica con la que dibujamos todo en pantalla mediante shaders programables.

3.2.2. Fundamento matemático

Para pasar un objeto 3D a una pantalla plana en 2D, se necesita multiplicar la posición de cada punto o vértice P por una serie de matrices dentro del *Vertex Shader*. La fórmula básica es la siguiente:

$$P' = M_{\text{proyección}} \cdot M_{\text{vista}} \cdot M_{\text{modelo}} \cdot P \quad (2)$$

Donde cada parte significa lo siguiente:

- P : Es la posición original del punto dentro de su propio modelo 3D.
- M_{modelo} : La matriz que acomoda el objeto en el mundo (su posición, rotación y tamaño).
- M_{vista} : La matriz que mueve todo según la posición y orientación de la cámara.
- $M_{\text{proyección}}$: La matriz que le da el efecto de perspectiva (lo que está lejos se ve más chico) y prepara todo para dibujarse en pantalla.
- P' : La posición final del punto, ya listo para proyectarse en la pantalla.

3.2.3. Implementación en código

Para enviar los datos de los objetos a la tarjeta de video, usamos los objetos de búfer (*VAO* y *VBO*). Aquí está el pedazo de código donde configuramos estos arreglos de memoria:

```
1 // Creamos y activamos el VAO y el VBO
2 unsigned int VAO, VBO;
3 glGenVertexArrays(1, &VAO);
4 glGenBuffers(1, &VBO);
5
6 glBindVertexArray(VAO);
7 glBindBuffer(GL_ARRAY_BUFFER, VBO);
```

```
8 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
9
10 // Le decimos a OpenGL cómo leer los datos de posición (x, y, z)
11 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
12 glEnableVertexAttribArray(0);
```

Listing 1: Configuración de los buffers (VAO, VBO) para dibujar la geometría.

3.2.4. Pasos que seguimos y resultados

1. **Modelado en Blender:** Modelamos el edificio y sus alrededores, asegurándonos de acomodar bien las superficies, normales y texturas.
2. **Exportar e importar:** Guardamos la escena como `.fbx` y la cargamos en el proyecto de C++ usando Assimp para separar las mallas y los puntos.
3. **Renderizado en OpenGL:** Conectamos los datos a la tarjeta de video con los VAO/VBO, le pasamos las matrices a los shaders y llamamos a las funciones de dibujo (`glDrawArrays` o `glDrawElements`).

3.3. Actividad 2: Variación - Personaje humanoide controlado con las flechas del teclado

El manual pide incorporar a la escena un personaje humanoide que avance y retroceda con las flechas Arriba y Abajo y que gire sobre el plano horizontal con Izquierda y Derecha. El proyecto base del laboratorio ya trae un personaje de ejemplo (`models/character.fbx`); para que la actividad no se limitara a volver a cargar ese mismo archivo, decidimos traer un personaje distinto desde Adobe Mixamo y adaptarlo nosotros a lo que el cargador del laboratorio es capaz de animar.

3.3.1. Selección y descarga del personaje en Mixamo

Elegimos al personaje **Exo Gray** y le aplicamos el clip **Walking**. Lo escogimos por encima del resto del catálogo porque es un humanoide sin armas ni accesorios separados del cuerpo, y porque su traje y su cara usan texturas distintas, lo que nos obligaba a resolver la adaptación completa y no solo el caso fácil. La animación se descargó con la casilla **In Place** activada: en la Práctica 04 comprobamos que, sin ella, toda la traslación del personaje vive en el nodo raíz, y aquí el desplazamiento lo tiene que controlar el programa con las flechas; si la animación también avanzara, el personaje se movería solo y regresaría a su punto de partida cada vez que el ciclo se reinicia. El resto de la configuración de descarga fue la misma de la práctica anterior: **FBX Binary (.fbx)**, *With Skin*, 30 cuadros por segundo y sin reducción de *keyframes*.

3.3.2. Lo que el cargador del laboratorio puede animar

Antes de cargar el archivo revisamos cómo funciona la clase `AnimatedModel` (`animatedmodel.h`) y encontramos tres restricciones que determinaron todo lo demás:

- La lista de huesos (`bones`) se vacía y se vuelve a llenar en cada `aiMesh` que procesa, y todas las mallas comparten el mismo arreglo `gBones`. Es decir, solo la última malla del archivo queda bien animada.
- Assimp separa en varias `aiMesh` una malla que tenga más de un material, así que dos materiales equivalen, para efectos del cargador, a dos mallas.

- El *shader* de *skinning* declara `uniform mat4 gBones[100]`, de modo que ningún índice de hueso puede pasar de 99.

El FBX de Exo Gray tal como sale de Mixamo incumple las tres: trae 8 mallas, 6 materiales y 112 huesos. El personaje de ejemplo, en cambio, es una sola malla de 2 456 vértices con 68 huesos, lo que explica por qué con él nunca se había notado ninguna de estas limitaciones.

3.3.3. Adaptación del personaje en Blender

En lugar de modificar la clase del laboratorio, que comparten todas las prácticas del curso, preferimos adaptar el modelo a ella, y lo hicimos con el guion `codigo/personaje_act3.py`, que corre en Blender sin interfaz:

1. Importa el FBX de Mixamo y elimina las tres mallas que no aportan forma: `EXO_Body` (24 vértices, un marcador del cuerpo), `EXO_HeadMask` (28 vértices) y `EXO_EyesSpec`, que es solo una capa de brillo sobre los ojos.
2. Las cinco mallas restantes usan dos texturas de color de 2048×2048 píxeles. Las pega lado a lado en un *atlas* de 4096×2048 y recorre las coordenadas UV de cada malla a la mitad que le corresponde, $u' = (u + k)/2$, con $k = 0$ para el traje y $k = 1$ para la cara y el cuerpo.
3. Deja un solo material con el atlas y une todas las mallas en una (*Ctrl+J*). El resultado es una malla de 16 738 vértices deformada por 94 huesos.
4. Borra de la armadura los 18 huesos que no tienen ningún peso en la malla, de modo que la armadura queda también con 94 huesos. Los 18 resultaron ser hojas del árbol (las puntas de los dedos, los párpados, `HeadTop_End`, `Throat` y las puntas de los pies), así que quitarlos no altera el movimiento de ningún otro hueso. El motivo de este paso se explica en la sección de experimentos.
5. Ajusta el rango de cuadros de la escena al de la acción (1 a 32) y exporta `Blender/ExoGray.fbx` con la animación horneada, sin huesos hoja adicionales y con la ruta de la textura reducida al nombre del archivo, que el cargador busca en la misma carpeta que el FBX.

```

1 bpy.context.view_layer.objects.active = arm
2 bpy.ops.object.mode_set(mode='EDIT')
3 for eb in list(arm.data.edit_bones):
4     if eb.name not in usados:           # huesos sin ningun peso en la malla
5         for hijo in eb.children:
6             hijo.parent = eb.parent
7             arm.data.edit_bones.remove(eb)
8 bpy.ops.object.mode_set(mode='OBJECT')
9 assert len(arm.data.bones) <= MAX_HUESOS      # gBones[100] del shader

```

Listing 2: Eliminación de los huesos sin pesos en `personaje_act3.py`.

3.3.4. Integración en OpenGL

En `codigo/05_ModelLoading.cpp` el personaje se carga con la misma clase y la misma escala de 0,01 que el de ejemplo, ya que ambos vienen de Mixamo con las unidades en centímetros. Conservamos los dos personajes y agregamos la tecla **G** para alternar entre ellos, porque así la variación queda a la vista dentro del mismo programa. También cambiamos el movimiento del código base, que avanzaba 0,005 unidades y giraba $0,5^\circ$ *por cuadro*: con esos valores la velocidad depende de los cuadros por segundo de cada computadora y, además, cruzar una ciudad de cien unidades tomaba varios minutos. Ahora ambas

cantidades se multiplican por `deltaTime`, con 6 unidades por segundo de avance y 90° por segundo de giro.

```

1 character = new AnimatedModel("models/character.fbx");
2 exoGray   = new AnimatedModel("models/ExoGray.fbx");
3 // ...
4 AnimatedModel* personaje = usarExo ? exoGray : character;
5 personaje->UpdateAnimation(deltaTime);
6 // ...
7 if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS)
8     position = position + scaleV * deltaTime * forwardView;
9 if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
10    rotateCharacter += giroV * deltaTime;
```

Listing 3: Carga de los dos personajes y avance independiente de los cuadros por segundo.

3.4. Actividad 3: Variación - Elementos geométricos alrededor del edificio

Para esta actividad partimos de la ciudad que se exportó en la Actividad 1 (Blender/Edificio.fbx), que es una sola malla de 191 880 vértices, y la rodeamos de calles, banquetas y casas modeladas también con primitivas y modificadores. Todo el procedimiento quedó escrito en el guion `codigo/alrededores_act4.py`, que corre dentro de la interfaz de Blender y va mostrando cada operación con su atajo de teclado; así cada paso se puede seguir en pantalla y quedó registrado en las capturas de la carpeta `Imágenes/E4`.

3.4.1. Trazo de la zona

Tomamos una decisión desde el inicio: las medidas de las manzanas no se escribieron a mano, sino que se calculan a partir de la caja que envuelve a la ciudad importada. La ciudad ocupa $x \in [-2, 0, 63, 1]$ y $y \in [-22, 2, 15, 8]$, y a partir de ahí el guion deja:

- Una banqueta de 6 unidades de margen alrededor de los edificios, que resulta en una manzana central de 79×51 unidades.
- Dos avenidas de 12 unidades de ancho, una enfrente y otra atrás de la manzana central, más dos calles laterales del mismo ancho.
- Una manzana de casas de 14 unidades de fondo del otro lado de cada avenida.

Las banquetas son cubos aplanados de 0.3 unidades de alto cuya cara superior queda en $z = 0$, a la misma altura que la base de los edificios, y el asfalto es un plano 0.3 unidades por debajo; ese escalón es lo que distingue la calle de la banqueta sin necesidad de usar colores. Las líneas centrales de las avenidas son un cubo delgado de $3 \times 0,5 \times 0,08$ unidades repetido con dos modificadores *Array*: uno con desplazamiento constante de 6 unidades en X (17 copias) y otro de 63 unidades en Y que lleva la misma fila a la avenida de atrás.

3.4.2. Modelado de la casa

La casa se construyó con el mismo procedimiento que la planta baja de la Actividad 1, para que toda la ciudad quedara hecha con la misma técnica:

1. **Cuerpo:** un cubo escalado a $6 \times 5 \times 3,5$ unidades.
2. **Retícula:** en Modo Edición se seleccionan los muros de enfrente y de atrás y se subdividen en tres columnas.

3. **Puertas:** a la columna central de cada uno de esos muros se le aplica un *Inset*, su cara interior se lleva a un hueco de $1,2 \times 2,2$ unidades a ras de piso y después se extruye 0.2 unidades hacia adentro.
4. **Ventanas:** el mismo *Inset* y *Extrude* sobre las dos columnas laterales y sobre los dos costados, con huecos de $1,2 \times 1,2$ unidades que empiezan a 1.2 unidades del piso.
5. **Techo:** un segundo cubo encima del cuerpo, con 0.3 y 0.4 unidades de alero, al que se le selecciona la cara superior y se escala a cero en *Y* (*S Y 0*). La cara se reduce a una arista y el cubo se convierte en un techo de dos aguas; los vértices que quedan encimados se funden con *Merge by Distance*.
6. **Unión:** cuerpo y techo se unen en un solo objeto (*Ctrl+J*).

Pusimos puerta en los dos muros largos por una razón práctica: la segunda fila de casas es una copia por *Array* de la primera, así que mira en la misma dirección; con puerta en ambos lados, cada fila queda con una puerta hacia su avenida sin tener que rotar ninguna copia.

3.4.3. Multiplicación y exportación

La casa se repite con dos *Array*: uno de 11.5 unidades en *X* que coloca 6 casas a lo largo de la manzana, y otro de 89 unidades en *Y* que repite la fila en la manzana de atrás; el número de casas también se calcula a partir del largo de la manzana. Al final se aplican todos los modificadores y las transformaciones (*Ctrl+A*), porque *model.h* ignora las transformaciones de los nodos del FBX, y se exporta *Blender/Ciudad.fbx* con la misma configuración que pide el manual para la Actividad 2: *Path Mode* en *Copy*, *Limit to Selected Objects* y *Triangulate Faces*.

3.4.4. Carga en OpenGL

El programa carga *Ciudad.fbx* en lugar del modelo de ejemplo, con la rotación de -90° en *X* que ya traía el código base para pasar de *Z* hacia arriba (Blender) a *Y* hacia arriba (OpenGL). El *shader* estático del laboratorio dibuja únicamente el color de la textura, y como la ciudad no trae textura escribimos un *shader* mínimo (*codigo/shaders/05_gris.vs* y *.fs*) que pinta el modelo de gris con una luz direccional; sin esa luz todas las caras tendrían el mismo tono y la ciudad se vería como una silueta plana. El personaje y la cámara arrancan sobre la avenida de enfrente, mirando hacia los edificios.

4. Experimentos

En esta sección se presentan las pruebas experimentales desarrolladas para validar el comportamiento del sistema gráfico bajo distintas configuraciones y parámetros.

4.1. Experimento 1: Construcción e Integración del Escenario Completo 3D

4.1.1. Objetivo del experimento

Comprobar que el escenario completo del edificio (modelado en Blender y exportado a formato FBX) pudiera cargarse correctamente en nuestra aplicación en C++ usando Assimp, verificando que la escala fuera la adecuada, que las texturas no se rompieran y que el rendimiento de fotogramas fuera fluido.

4.1.2. Pruebas exitosas

Una vez corregidos los detalles del código, el escenario en 3D se cargó de manera estable. Pudimos navegar libremente por el mapa usando el teclado y el ratón sin caídas feas en la tasa de refresco, manteniéndose a unos 60 FPS bien fluidos. Las texturas de las paredes y las proporciones generales del edificio se mostraron respetando el diseño original que teníamos en Blender.

4.1.3. Pruebas fallidas y análisis de errores

En los primeros intentos las cosas no salieron a la primera y nos topamos con un problema fuerte:

- **Solo cargaba el personaje:** Al ejecutar la animación para probar la escena integrada, nos encontramos con la sorpresa de que únicamente aparecía el personaje caminando en la nada (Figura 1); los edificios y las torres que habíamos modelado previamente simplemente no se veían en la pantalla.

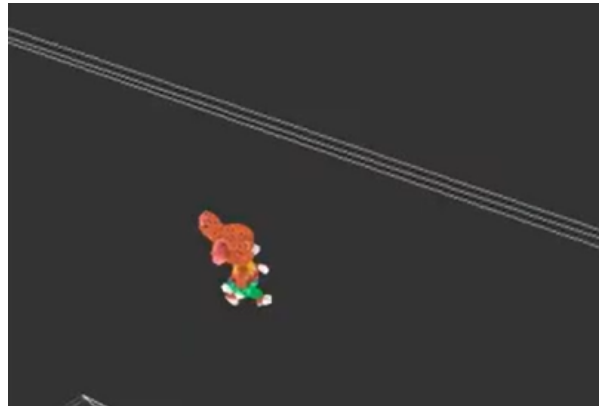


Figura 1: Primera carga del escenario: Solo se despliega el personaje sobre el plano.

Análisis del error: Al revisar qué había pasado, descartamos rápido un fallo en el código de C++ o en los shaders, ya que la aplicación compilaba bien y el personaje sí se renderizaba. El verdadero problema estuvo en el proceso de exportación desde Blender.

La causa fue que al exportar la escena a formato `.fbx`, se nos pasó seleccionar todos los nodos independientes que conformaban la ciudad (los bloques de edificios, torres y avenidas). Al no marcar la opción de exportar la selección completa o la escena activa, la librería Assimp leyó un archivo vacío. La solución fue simple pero importante: regresar a Blender, seleccionar absolutamente todos los objetos de la escena (Tecla **A**) y volver a exportar asegurando que toda la jerarquía de mallas quedara empaquetada en el mismo archivo.

4.2. Experimentos de la Actividad 2: segundo personaje de Mixamo

4.2.1. Objetivo del experimento

La variación de esta actividad fue sustituir el personaje de ejemplo del laboratorio por uno descargado por nosotros (Exo Gray) y comprobar qué tanto hay que adaptar un modelo de Mixamo para que lo anime el mismo cargador. El personaje de ejemplo se conservó en el programa para comparar los dos directamente con la tecla **G**.

4.2.2. Pruebas fallidas y análisis de errores

Piernas colapsadas. La primera versión ya tenía una sola malla y un solo material, y aun así, al ejecutar el programa, el torso, los brazos y la cabeza se veían y se animaban bien, pero las dos piernas salían convertidas en picos que se estiraban desde la rodilla hasta un punto en el piso (Figura 2). Lo primero que descartamos fueron los pesos: revisamos en Blender las influencias de cada vértice y ninguno tenía más de cuatro huesos, y en todos los pesos sumaban exactamente 1.

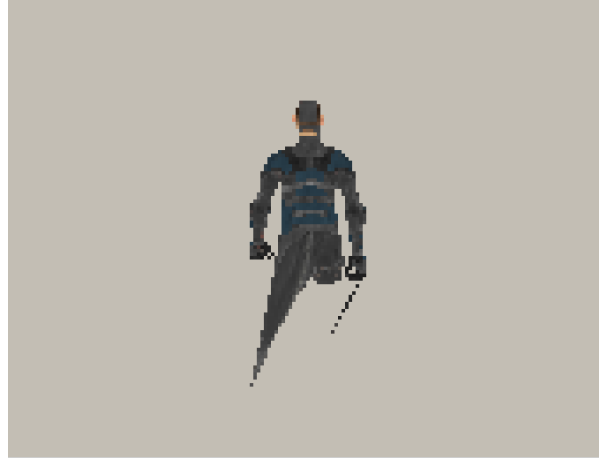


Figura 2: Primera carga de Exo Gray en OpenGL (acercamiento): la parte superior del cuerpo se anima correctamente, pero los vértices de las piernas se colapsan hacia el origen del modelo.

La pista estuvo en la forma del error: un vértice multiplicado por una matriz nula termina en el origen del modelo, que en un personaje de Mixamo está justo entre los pies. Al listar la armadura por índice encontramos que el exportador FBX de Blender escribe un *cluster* por cada hueso de la armadura, tenga pesos o no; con 112 huesos, los de las piernas (*LeftUpLeg* a *RightToeBase*) quedaban con los índices 102 a 110, fuera del arreglo `gBones[100]` del *shader*. La parte superior funcionaba simplemente porque sus huesos aparecen antes en el árbol. La corrección fue borrar los 18 huesos sin pesos, todos hojas, con lo que la armadura bajó a 94 huesos y las piernas a índices menores que 100.

Animación congelada. En la misma prueba la consola reportó **Animation total frames: 249** para Exo Gray, cuando el clip de Mixamo tiene 32 cuadros. El exportador hornea la animación sobre el rango de la escena de Blender, que por omisión va de 1 a 250, así que el personaje daba un ciclo de caminata y se quedaba quieto en la última pose durante unos siete segundos antes de reiniciar. Se corrigió ajustando el rango de la escena al de la acción antes de exportar; después de eso la consola reporta 31 cuadros para los dos personajes.

4.2.3. Pruebas exitosas

Con las dos correcciones, Exo Gray se carga como una sola malla de 16 738 vértices, con su textura completa, y camina sin deformaciones. Como comparación, el personaje de ejemplo tiene 2 456 vértices y 68 huesos. La diferencia de tamaño sí se nota en un aspecto: `AnimatedModel` asigna los pesos recorriendo, para cada vértice, todos los pesos de todos los huesos, por lo que el tiempo de carga crece con el producto de ambos, y con Exo Gray el programa tarda alrededor de medio minuto en mostrar la escena.

4.3. Experimentos de la Actividad 3: zona ampliada con casas, calles y banquetas

4.3.1. Objetivo del experimento

La variación de esta actividad fue generar los alrededores a partir de las medidas de la ciudad en lugar de colocarlos a mano, para verificar que el mismo procedimiento se adapta a una ciudad de otro tamaño sin rehacer el modelado.

4.3.2. Pruebas exitosas

El guion se escribió primero sobre una versión preliminar de la ciudad que ocupaba $x \in [-2, 2, 72]$ y $y \in [-20, 4]$, y con ella dejaba 7 casas por fila. Al cambiar al modelo definitivo de la Actividad 1, que es más angosto en X y bastante más profundo en Y , no fue necesario tocar ninguna medida: el guion recalculó las manzanas a 79×51 unidades, bajó el número de casas a 6 por fila, ajustó la separación del segundo *Array* de las casas a 89 unidades y la de las líneas de las avenidas a 63 unidades. Las casas, con sus puertas y ventanas, no cambiaron en absoluto, porque su modelado no depende del tamaño de la ciudad.

4.3.3. Pruebas fallidas y análisis de errores

En la primera ejecución en OpenGL la mayor parte de la ventana quedó cubierta por una superficie gris clara y el personaje no aparecía (Figura 3). La cámara se había colocado a 6 unidades de altura y a $z = 50$, que en la ciudad ampliada cae justo sobre la fila de casas de enfrente; como el techo de las casas llega exactamente a 6 unidades, la cámara quedó dentro de uno de ellos. La corrección fue subirla a 10 unidades de altura e inclinarla 20° hacia abajo, de modo que queda por encima de los techos y sigue viendo al personaje sobre la avenida.



Figura 3: Primera ejecución de la zona ampliada en OpenGL: la cámara quedó dentro del techo de una casa de la fila de enfrente y el techo cubre casi toda la vista.

5. Resultados

A continuación se exponen y analizan los resultados obtenidos en cada una de las actividades prácticas y pruebas experimentales.

5.1. Resultados de la Actividad 1: Construcción e Integración del Escenario Completo 3D

El desarrollo de la actividad se completó con éxito dividiéndose en tres etapas, representadas en las siguientes figuras:

5.1.1. 1. Modelado base en Blender

En la Figura 4 se muestra el diseño original del entorno urbano construido en Blender. En esta etapa se distribuyeron las pirámides, edificios y torres a lo largo de lo que sería la avenida principal para establecer las proporciones y la disposición espacial antes de la exportación.

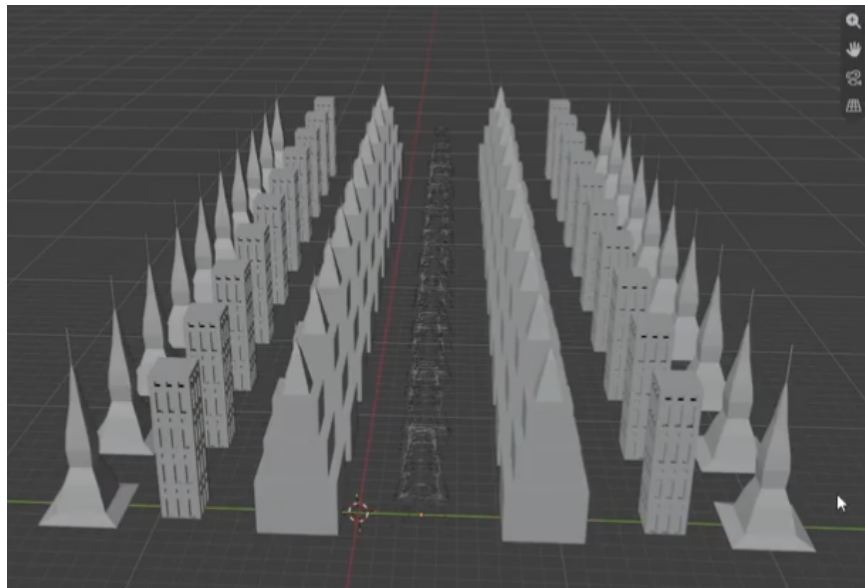


Figura 4: Modelado y distribución espacial del escenario urbano realizado en Blender.

5.1.2. 2. Importación y estructura de malla en OpenGL

En la Figura 5 se aprecia la escena cargada dentro de la aplicación C++ mediante la librería Assimp, mostrada en modo *wireframe* (malla de alambre). Esto nos permitió verificar que la geometría, las avenidas con sus líneas divisoras y el volumen de cada edificio se leyeran e interpretaran de manera correcta en el búfer de la GPU sin deformaciones en los vértices.

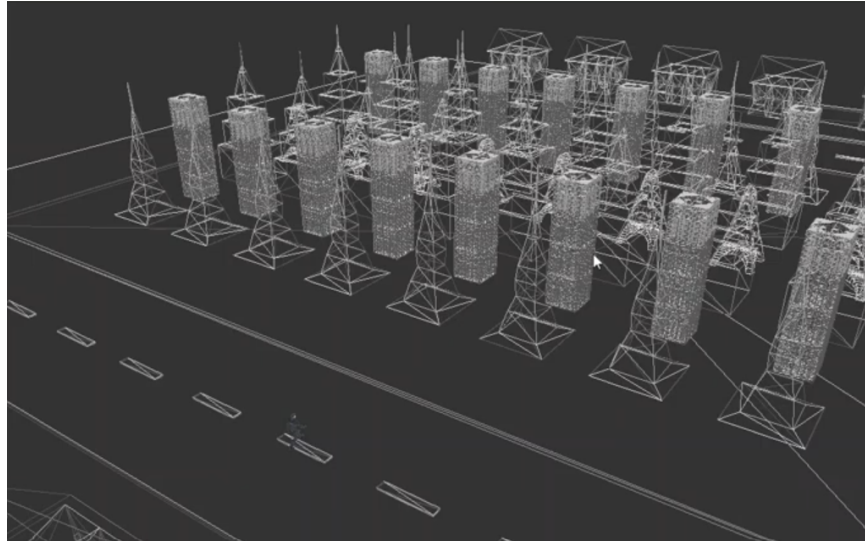


Figura 5: Visualización de la malla (*wireframe*) del escenario importado y renderizado en OpenGL.

5.1.3. 3. Integración final con el personaje en movimiento

Finalmente, en la Figura 6 se observa el resultado integrado: el escenario completo de fondo conviviendo con el personaje animado en primer plano sobre la calle. El personaje ejecuta su ciclo de movimiento fluido mientras la cámara mantiene la perspectiva adecuada para recorrer la ciudad.

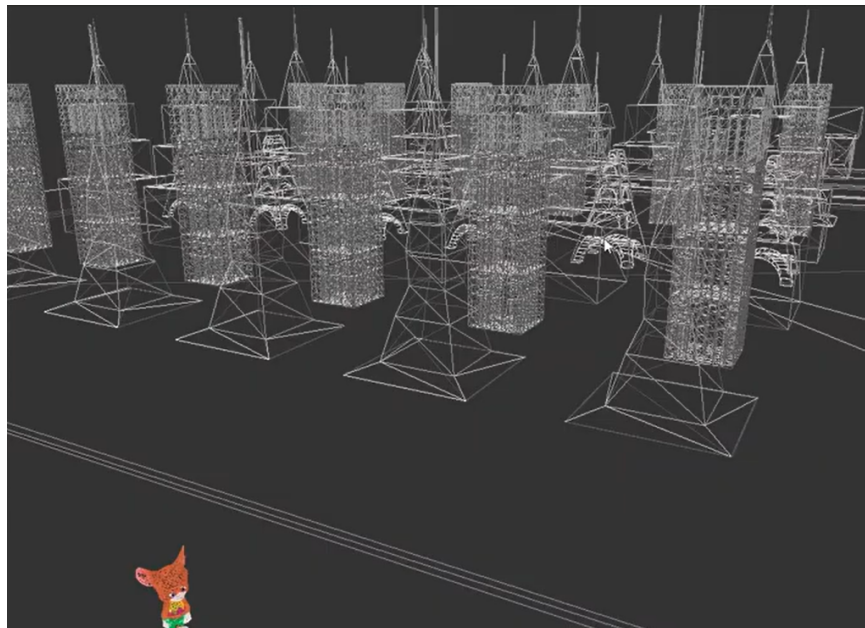


Figura 6: Resultado final en OpenGL: escenario tridimensional completo con el personaje animado en movimiento.

5.1.4. Análisis de ventajas y desventajas

- **Ventajas:**

- **Eficiencia de renderizado:** El uso de objetos de búfer (*VAO* y *VBO*) permite transferir la masa de datos geométricos de la ciudad a la memoria de la tarjeta de video una sola vez, manteniendo una tasa de cuadros fluida durante la animación.
 - **Verificación en tiempo real:** Renderizar en modo *wireframe* durante el desarrollo facilita detectar errores de geometría, normales invertidas o superposición de mallas rápidamente.
- **Desventajas:**
- **Carga de vértices elevados:** Al tener múltiples edificios detallados en la escena, la densidad de la malla aumenta significativamente, lo que exige mayor cómputo de transformaciones matriciales en el *Vertex Shader*.
 - **Control manual de punteros:** Se requiere un manejo estricto en el mapeo de atributos de vértices e índices de huesos para evitar desajustes visuales o fallos en la animación del personaje.

5.2. Resultados de la Actividad 2: Personaje humanoide controlado con las flechas del teclado

La adaptación en Blender dejó a *Exo Gray* como un modelo que *AnimatedModel* puede animar sin modificaciones: una malla, un material y 94 huesos. La Figura 7 muestra el atlas resultante, con el traje en la mitad izquierda y la cara y el cuerpo en la derecha, y la Figura 8 muestra el modelo ya unido y animado dentro de Blender antes de exportarlo. Que la cara y el traje se vean en su lugar confirma que el recorrido de las coordenadas UV a cada mitad del atlas fue correcto.

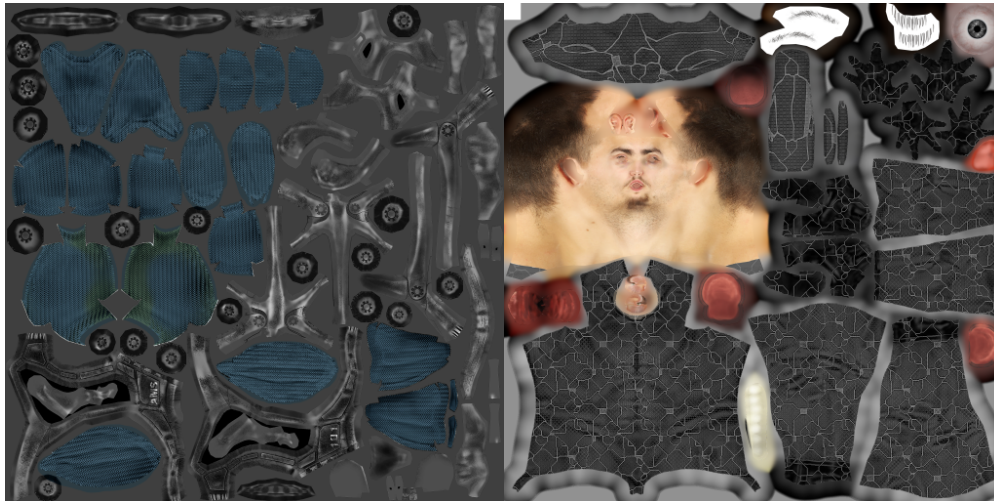


Figura 7: Atlas de 4096×2048 píxeles generado por `personaje_act3.py`: a la izquierda la textura del traje y a la derecha la de la cara y el cuerpo.



Figura 8: Exo Gray en Blender después de la adaptación: una sola malla de 16 738 vértices con el atlas aplicado, a mitad del ciclo de caminata.

En OpenGL el personaje aparece sobre la avenida de enfrente de la ciudad y responde a las cuatro flechas (Figura 9): las flechas **Arriba** y **Abajo** lo desplazan en la dirección hacia la que mira y las laterales lo giran sobre el eje *Y*, mientras la caminata se reproduce en su lugar. Con la tecla **G** se cambia al personaje de ejemplo del laboratorio (Figura 10), que se mueve con exactamente los mismos controles. Como los dos personajes comparten la posición y la orientación, el cambio ocurre en el mismo punto de la escena.

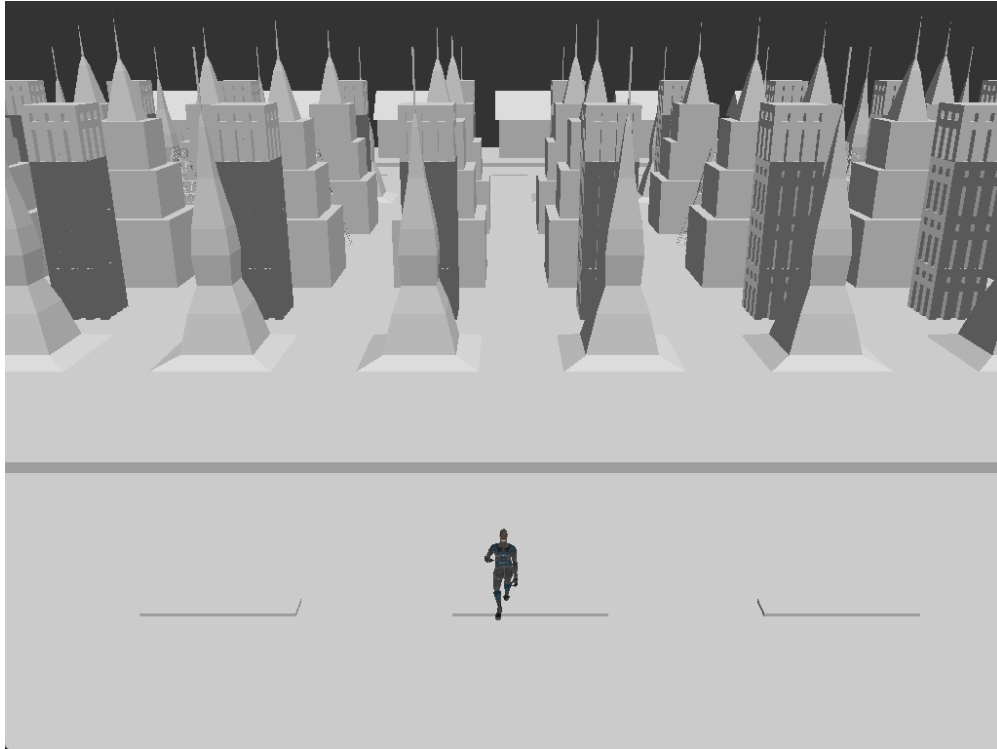


Figura 9: Exo Gray cargado en OpenGL con `AnimatedModel`, sobre la avenida de enfrente de la ciudad.

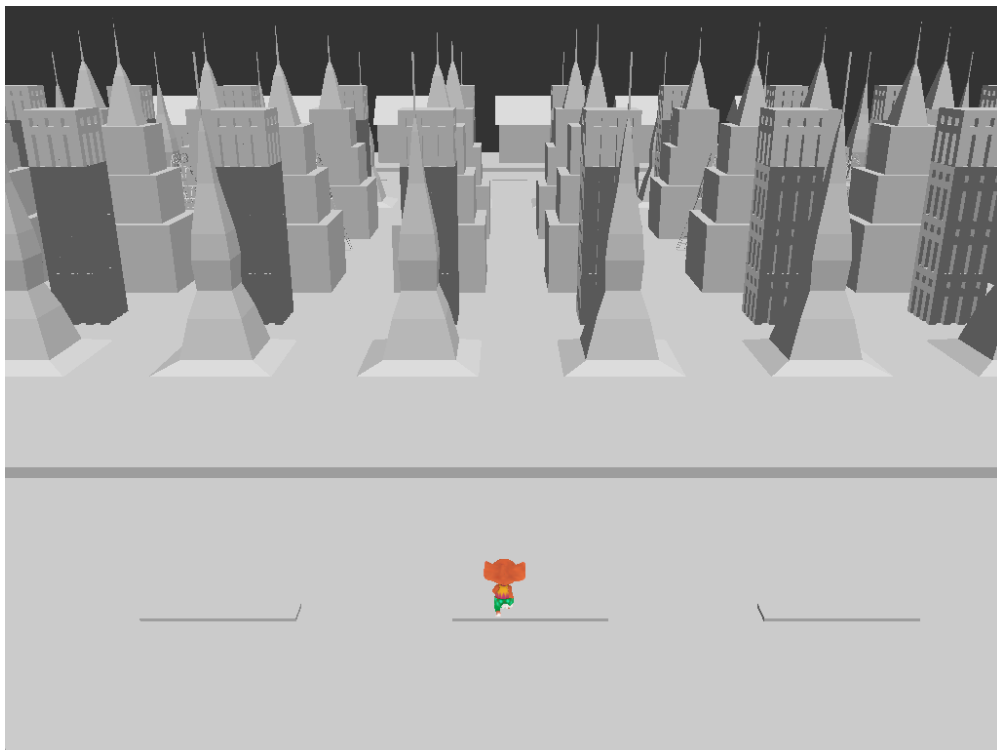


Figura 10: El personaje de ejemplo del laboratorio (`character.fbx`) en la misma posición, después de presionar la tecla G.

5.2.1. Análisis de ventajas y desventajas

- **Ventajas:** Adaptar el modelo en lugar del cargador deja intacta la clase que usan las demás prácticas, y como todo el proceso quedó en un guion, cualquier otro personaje de Mixamo se puede preparar con el mismo procedimiento cambiando solo el archivo de entrada.
- **Desventajas:** El atlas pesa 12 MB y, al juntar las texturas en una sola, cada parte del cuerpo dispone de la mitad de la resolución horizontal que tenía. Además, el tiempo de carga crece mucho con el número de vértices por la forma en que la clase asigna los pesos.

5.3. Resultados de la Actividad 3: Elementos geométricos alrededor del edificio

La secuencia del modelado quedó registrada en Blender paso a paso. La Figura 11 resume la construcción de la casa: los muros subdivididos en tres columnas, las puertas y ventanas hundidas con *Inset* y *Extrude*, y el techo de dos aguas obtenido al escalar a cero la cara superior de un cubo.

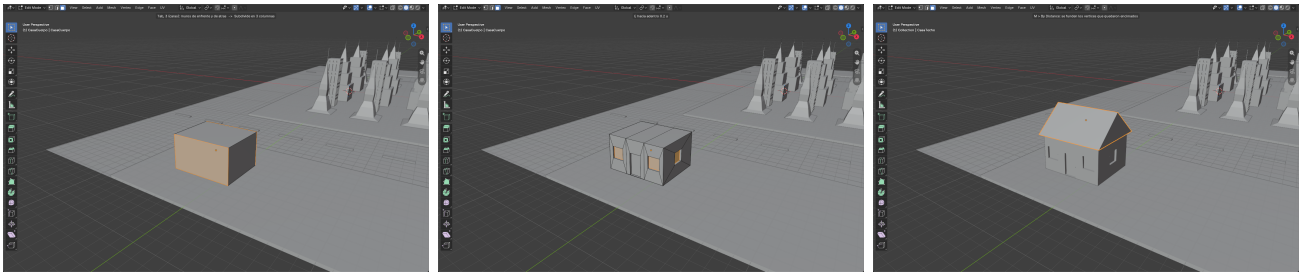


Figura 11: Construcción de la casa: subdivisión de los muros en tres columnas (izquierda), puertas y ventanas hundidas (centro) y techo de dos aguas con *S Y 0* (derecha).

La Figura 12 muestra el modelo completo antes de exportarlo: la ciudad de la Actividad 1 sobre su banqueta, las dos avenidas con sus líneas centrales y las dos filas de seis casas, cada una con la puerta hacia su avenida. El archivo exportado, *Ciudad.fbx*, pesa 6.3 MB.

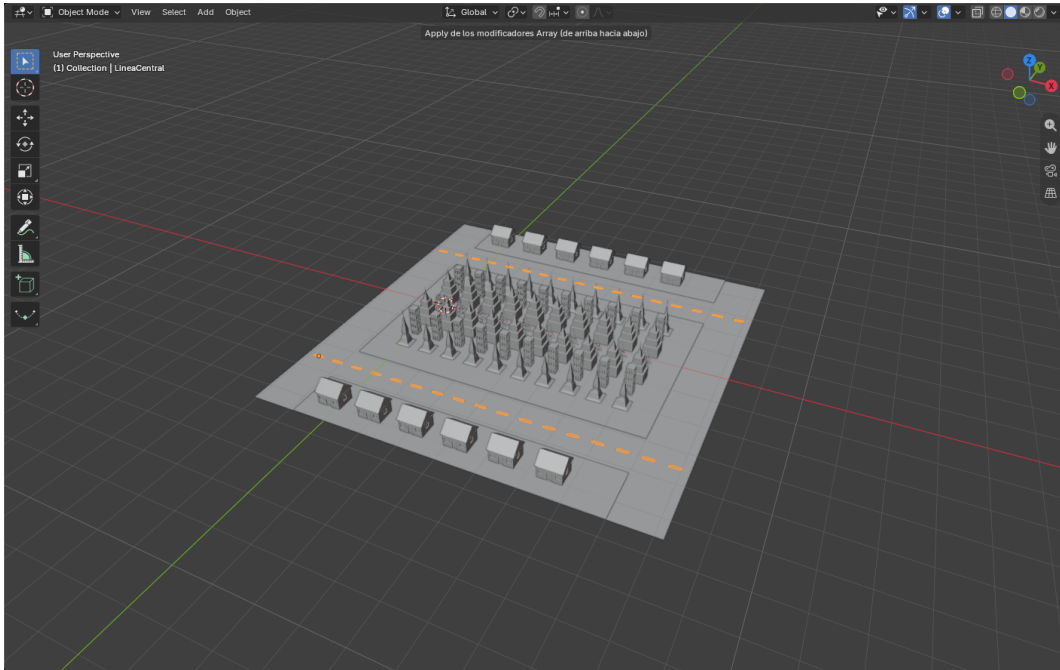


Figura 12: Zona ampliada en Blender: manzana central con los edificios, avenidas de enfrente y de atrás con sus líneas centrales y dos filas de casas.

Cargado en OpenGL, el modelo se ve completo desde la avenida (Figura 13). Sin usar colores, la diferencia de altura entre banqueta y asfalto y la luz direccional del *shader* gris bastan para distinguir la calle, el borde de la banqueta y las caras de cada edificio. Al recorrer la escena con el personaje, la avenida y las calles laterales permiten rodear toda la manzana de los edificios.

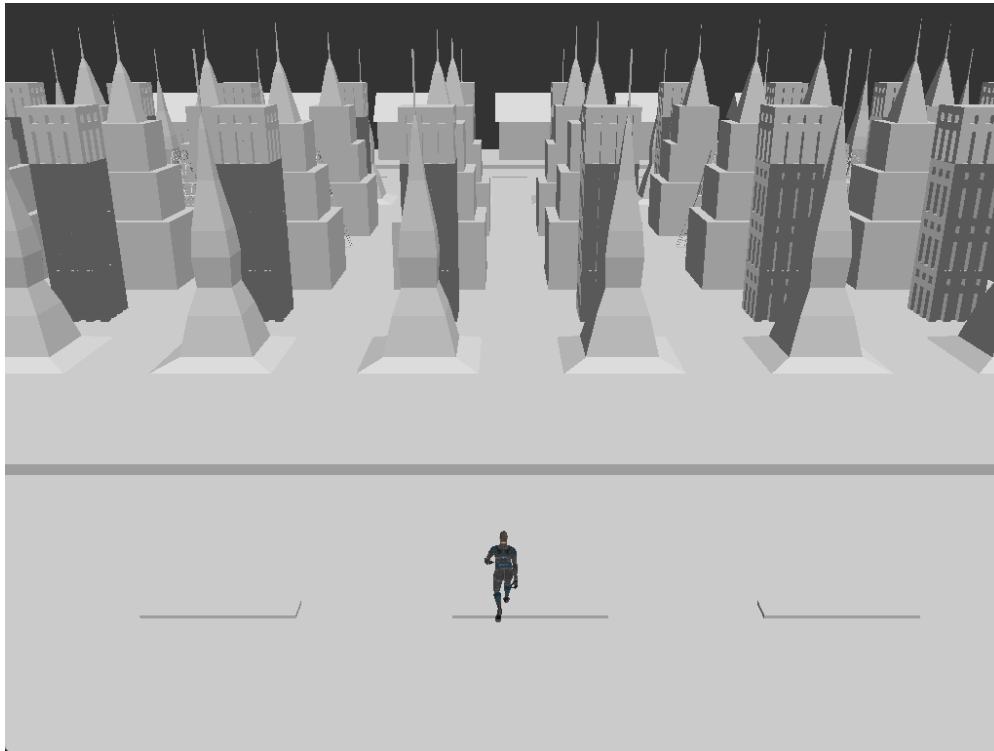


Figura 13: Ciudad.fbx en OpenGL con el *shader* gris, vista desde la avenida de enfrente.

5.3.1. Análisis de ventajas y desventajas

- **Ventajas:** Al calcular el trazo desde la caja de la ciudad, el procedimiento se adaptó sin cambios a un modelo de otro tamaño, y el uso de *Array* hace que agregar o quitar casas sea cuestión de cambiar un solo número.
- **Desventajas:** Como todo se exporta en un solo FBX con un solo material, en OpenGL la ciudad, las casas y las calles comparten el mismo *shader* y no pueden tratarse por separado; además, las casas son copias idénticas, así que la zona se ve repetitiva.

6. Conclusiones Individuales

321137135: Basilio Illescas Andrés

Los objetivos de esta entrega se cumplieron: la ciudad modelada en Blender quedó exportada a FBX y cargada en OpenGL, rodeada de calles, banquetas y casas hechas con las mismas primitivas y modificadores, y el recorrido se puede hacer con un personaje que no es el de ejemplo del laboratorio sino uno que descargamos y adaptamos nosotros. Lo que realmente me llevo de la práctica es que el formato de intercambio no garantiza nada por sí solo. Que Exo Gray estuviera en FBX, igual que el personaje de ejemplo, no significaba que el cargador lo fuera a entender: tenía ocho mallas, seis materiales y ciento doce huesos, y la clase `AnimatedModel` solo sabe animar una malla con cien huesos como máximo. Hasta ese momento el cargador «funcionaba», pero en realidad nunca se le había pedido nada fuera del caso fácil.

La lección de método vino de la falla de las piernas. El síntoma invitaba a buscar en el lugar equivocado, porque unas piernas deformadas hacen pensar de inmediato en los pesos, y si me hubiera quedado con esa idea habría pasado horas repintando influencias que estaban bien. Lo que resolvió el problema fue leer la

forma del error: todos los vértices de las piernas iban a parar al mismo punto entre los pies, que es justo donde termina un vértice multiplicado por una matriz nula. A partir de ahí bastó listar los huesos por índice para encontrar que los de las piernas caían entre el 102 y el 110, fuera del arreglo de cien matrices del *shader*, y que la parte de arriba funcionaba solo porque sus huesos aparecen primero en el árbol.

Si en la Práctica 03 me quedé con que «se ve bien» no equivale a «está bien», y en la 04 con que dos cosas que se ven iguales no necesariamente lo son, aquí me queda una tercera versión: que algo funcione con un ejemplo no demuestra que funcione en general. El personaje de ejemplo, con una sola malla y sesenta y ocho huesos, era justamente el caso que ocultaba las tres limitaciones del cargador, y solo al traer un modelo distinto aparecieron todas al mismo tiempo. Algo parecido pasó con la ampliación de la ciudad: como el trazo de las manzanas se calcula a partir de la caja de los edificios, el mismo guion se adaptó sin cambios cuando pasamos al modelo definitivo de la Actividad 1, y esa fue la prueba de que el procedimiento era general y no un acomodo hecho a la medida de una sola escena.

424065465: Cruz Macedo Samuel Santiago

Entrando a esta práctica pensé que lo laborioso iba a ser modelar en Blender y que pasar el modelo a OpenGL sería solo exportar y cargar un archivo. Fue al revés: el modelado se hace con las mismas primitivas y modificadores que ya conocíamos, y el trabajo de verdad estuvo en lograr que lo que se ve en Blender sea lo mismo que dibuja nuestro programa. Lo primero que me hizo ver esto fue la orientación: Blender trabaja con *Z* hacia arriba y OpenGL con *Y* hacia arriba, así que sin la rotación de -90° en *X* la ciudad aparecería acostada. Y lo segundo fue que `model.h` ignora las transformaciones de los nodos del FBX, de modo que si un objeto se mueve o se escala en Blender sin aplicar la transformación, en OpenGL aparece en su posición original como si nunca se hubiera tocado.

Lo que más me sorprendió fue el error de las piernas de Exo Gray. El archivo cargaba sin marcar ningún error, el torso caminaba bien y las piernas salían convertidas en picos clavados en el piso. La causa no estaba en el modelo ni en la animación sino en un número: el *shader* reserva espacio para cien matrices de huesos y los de las piernas tenían índices arriba de cien. En la Práctica 03 ya me había tocado ver un programa que corre sin quejarse y aun así entrega algo indefinido, y aquí volvió a pasar, solo que ahora del lado de la GPU, donde leer fuera de un arreglo no detiene nada, simplemente regresa basura.

De la Actividad 4 me quedo con lo útil que resultó no usar colores. Al principio parecía que, sin ellos, la calle y la banqueta se iban a confundir, pero bastó dejar el asfalto 0.3 unidades más abajo y pintar el modelo con una sola luz direccional para que cada escalón y cada cara se distinguieran. Hasta entonces asociaba el color con lo que hace legible una escena, y resultó que la geometría y la iluminación cargan con casi todo el trabajo. Considero que los objetivos se cumplieron, y me queda claro que esto es la base de lo que sigue, porque una vez que el puente entre Blender y OpenGL funciona, cualquier escena del proyecto final va a llegar por este mismo camino.

321258173: Medel Piña Ángel de Jesús

Aterrizar el puente entre el software de modelado 3D y el motor gráfico en OpenGL fue el mayor aprendizaje de esta práctica. El desarrollo demostró que exportar un archivo FBX no es un procedimiento automático, ya que el motor gráfico impone restricciones estrictas sobre la estructura de los datos que puede procesar. Lo más revelador fue comprender cómo la biblioteca Assimp descompone los modelos: cada material asignado en Blender se traduce en una submalla independiente (*submesh*). Para que la clase `AnimatedModel` del laboratorio pudiera interpretar a Exo Gray sin corromper la animación, fue indispensable unificar la geometría en una sola malla mediante un atlas de texturas de 4096×2048 y remapear matemáticamente las coordenadas UV ($u' = (u + k)/2$), resolviendo la compatibilidad desde la preparación del modelo.

En el diseño del entorno urbano para la Actividad 4, el aspecto más enriquecedor fue aplicar un enfoque paramétrico en lugar de uno estático. Generar las banquetas, las calles y la distribución de las casas a partir de las dimensiones de la caja envolvente (*bounding box*) del edificio central garantizó que el escenario se adaptara de forma automática ante cualquier variación de escala. Asimismo, el error inicial de la cámara dentro del techo de una casa evidenció la importancia de planificar la altura y orientación del punto de vista en función de las cotas máximas de los objetos circundantes; elevarla a 10 unidades e inclinarla 20° hacia abajo permitió una supervisión clara de toda la avenida.

Por último, la implementación del sistema de navegación interactiva demostró la necesidad de desvincular el movimiento de la tasa de cuadros por segundo mediante `deltaTime`. Configurar la traslación a 6 unidades por segundo y el giro a 90° por segundo proporcionó un desplazamiento fluido y consistente para recorrer la ciudad, mientras que la posibilidad de alternar en tiempo real entre el personaje de ejemplo y Exo Gray con la tecla `G` confirmó que la cinemática y la posición espacial operan de forma completamente desacoplada de la geometría visual.

321150503: Pérez Olvera Alexis Abraham

El desarrollo de esta práctica permitió comprender a fondo los mecanismos de bajo nivel involucrados en la carga, adaptación y renderizado de mallas tridimensionales complejas dentro de OpenGL. Trabajar directamente con la biblioteca Assimp y los búferes de vértices (VAO y VBO) dejó en claro que la integración de recursos 3D requiere una correspondencia exacta entre la topología del archivo exportado y las estructuras de datos esperadas por el *pipeline* gráfico, especialmente cuando se implementa animación por esqueletos (*skeletal skinning*) acelerada en la GPU.

El análisis y resolución del error en las extremidades de Exo Gray constituyó la experiencia técnica más significativa de la práctica. Al observar que el torso se animaba correctamente mientras las piernas colapsaban en picos hacia el suelo, se evidenció el comportamiento de la GPU ante accesos fuera de rango: el *shader* reservaba espacio para un máximo de cien matrices (`uniform mat4 gBones[100]`), pero el archivo FBX contenía 112 huesos debido a nodos terminales sin influencias, dejando los huesos de las piernas con índices mayores a 99. En GLSL, leer fuera de los límites de un arreglo no genera excepciones en tiempo de ejecución, sino que devuelve datos nulos o indefinidos; al multiplicar la posición de los vértices por matrices en ceros, estos colapsaron matemáticamente hacia el origen (0,0,0) del modelo. Purgar los 18 huesos innecesarios en Blender redujo el esqueleto a 94 huesos y solucionó la deformación, demostrando la necesidad crítica de respetar las limitaciones de memoria de los *shaders* desde la fase de exportación.

Adicionalmente, la implementación del *shader* con iluminación direccional para la zona urbana en la Actividad 4 subrayó el papel de los vectores normales en la percepción espacial. Sin componentes de iluminación, una geometría compleja sin texturas pierde toda definición volumétrica en pantalla; el cálculo difuso de Lambert, sumado al desnivel físico de 0.3 unidades entre el asfalto y la banqueta, bastó para dotar de profundidad y legibilidad al entorno. En conclusión, la práctica cumplió plenamente sus objetivos al demostrar que la carga de modelos no se limita a leer archivos en disco, sino a coordinar la geometría, las matrices de transformación y los recursos de la GPU en un sistema coherente.

320224159: Segura Cedeño Luisa María

Los objetivos de esta práctica se cumplieron con éxito, ya que logramos llevar a cabo todo el flujo de trabajo para cargar y adaptar modelos 3D complejos en OpenGL. Lo esperado era construir un escenario tridimensional completo, integrar personajes animados controlados desde el teclado y ambientar los alrededores de forma fluida y funcional, y al final todo se logró exitosamente.

A lo largo del desarrollo entendí lo importante que es preparar bien los modelos desde el software de diseño antes de pasarlos a C++. En lo personal, lo que más me gustó e impresionó de todo el proceso

fue utilizar la modificación Array en Blender. Fue una parte fundamental para la creación de la ciudad, ya que nos permitió duplicar los elementos de una manera superordenada. Gracias a esto logramos que la ciudad se viera mucho más grande, con edificios más altos y un toque más realista, optimizando tiempo y esfuerzo en el modelado. En conclusión, la práctica me dejó una excelente experiencia sobre cómo conectar el diseño 3D con la programación de gráficos en tiempo real.

7. Enlace al Video Demostrativo

A continuación se proporciona el enlace al video demostrativo de la práctica, en el cual se expone de forma detallada el funcionamiento, compilación y compendio de resultados de cada una de las actividades realizadas:

Video Demostrativo

Enlace al funcionamiento de las actividades de la práctica:

<https://youtu.be/PY0Z3PYqdog>

Referencias Bibliográficas

- [1] Hearn, D., Baker, M. P., & Carithers, W. (2011). *Computer Graphics with OpenGL* (4.^a ed.). Pearson Prentice Hall.
- [2] Foley, J. D., Van Dam, A., Feiner, S. K., & Hughes, J. F. (1996). *Computer Graphics: Principles and Practice in C* (2.^a ed.). Addison-Wesley Professional.
- [3] de Vries, J. (2020). *Learn OpenGL: Learn modern OpenGL graphics programming in a step-by-step fashion*. Recuperado de <https://learnopengl.com/>
- [4] Khronos Group. (s.f.). *OpenGL 3.3 Reference Pages*. The Khronos Group Inc. Recuperado de <https://registry.khronos.org/OpenGL-Refpages/>
- [5] Teodoro-Vite, S., et al. (2019). *Fundamentos y Técnicas del Cómputo Gráfico*. Facultad de Ingeniería, Universidad Nacional Autónoma de México.